PROGRAM 5

Working with Vector Database-1 (Working with Collections)

Title: Working with vector database-1(working with collections)

Problem statement: Write a python program to perform following operations on collection

- Create collection
- Add data and display
- Query the collection
- Delete the data

Method:

1. Install chromadb using pip install command and import it.
2. Create chroma client
3. Create collections and perform all operations on collection

Method

Step 1: Check Python Version
Open the VS Code terminal and type:

py --version

Expected output: Python 3.14

Step 2: Install ChromaDB

py -m pip install --upgrade chromadb

Step 3: Create Python File
Create a file named prg5.py.

Complete Program

```
# 1: Import ChromaDB and create an in-memory client.
import chromadb
client = chromadb.Client()
# 2: Create a collection for student documents.
collection = client.create_collection(
 name="student_collection"
)
# 3: Add documents with IDs and descriptive metadata.
# Items at the same position belong to the same record.
# ChromaDB generates document vectors automatically.
collection.add(
 ids=["1", "2", "3", "4"],
 documents=[
 "Python is a programming language",
 "Java is an object oriented language",
 "SQL is used to manage databases",
 "Machine Learning is a branch of Artificial Intelligence"
 ],
 metadatas=[
 {"topic": "Programming", "subject": "Python"},
 {"topic": "Programming", "subject": "Java"},
 {"topic": "Database", "subject": "SQL"},
 {"topic": "AI", "subject": "Machine Learning"}
 ]
)
print("Records added:", collection.count())
# 4: Retrieve documents, metadata and stored vectors.
# Embeddings must be requested explicitly using include.
data = collection.get(
 include=["documents", "metadatas", "embeddings"]
)
# 5: Display the complete vector for each document.
# zip() groups corresponding fields into one record.
for record_id, document, metadata, vector in zip(
 data["ids"], data["documents"],
 data["metadatas"], data["embeddings"]
):
```

```
 print("\nID:", record_id)
 print("Document:", document)
 print("Metadata:", metadata)
 print("Vector dimensions:", len(vector))
 print("Vector:", vector.tolist())
# 6: Search for the two most similar documents.
# Query results contain a separate list for each query.
result = collection.query(
 query_texts=["programming language"],
 n_results=2,
 include=["documents", "metadatas", "distances"]
)
print("\n--- Similarity search ---")
for record_id, document, metadata, distance in zip(
 result["ids"][0], result["documents"][0],
 result["metadatas"][0], result["distances"][0]
):
 print("ID:", record_id)
 print("Document:", document)
 print("Metadata:", metadata)
 print("Distance:", distance)
# 7: Retrieve records using a metadata filter.
filtered = collection.get(
 where={"topic": "Programming"},
 include=["documents", "metadatas"]
)
print("\n--- Programming records ---")
print(filtered)
# 8: Delete the Java record using its ID.
collection.delete(ids=["2"])
print("\nRecords after deletion:", collection.count())
# Step 9: Display the remaining documents and metadata.
print(collection.get(include=["documents", "metadatas"]))
```

Step 6: Run the Program

python prg5.py